

MediControl

Aplicacion de Gestion de Medicamentos



Trabajo Practico Python

Introduccion a la Algoritmia — UADE

Cristian Aostri Lopez

David Espinosa

Facundo Balmacena

Patricio Zermo

Python | OpenAI GPT-4o | python-dotenv

1. Portada
2. Contenido
3. Descripcion del proyecto y estructura del programa
4. Menu principal — 10 opciones
- 5.Codigo: agregar medicamento y validaciones
6. Codigo: alertas personalizadas
7. Codigo: verificar alertas y calcular dias
8. Codigo: asistente IA (GPT-4o)
9. Codigo: main() y conclusion

Descripcion del Proyecto

Que hace MediControl y como esta estructurado

Que es MediControl?

MediControl es un programa de consola en Python que permite registrar, consultar y administrar medicamentos del hogar. Funciona sin internet, sin registro y sin instalaciones adicionales. Esta pensado especialmente para adultos mayores y personas con enfermedades cronicas.

Librerias utilizadas

datetime

Manejo de fechas de inicio, termino y vencimiento

openai

Conexion con GPT-4o para el asistente IA

dotenv

Carga segura de la API key desde archivo .env

os

Acceso a variables de entorno del sistema

Funciones del programa

agregar_medicamento()

Alta de nuevo medicamento con validaciones

mostrar_medicamentos()

Lista todos los medicamentos registrados

registrar_toma()

Descuenta stock al registrar una toma

ingresar_alerta()

Crea alertas personalizadas con fecha y prioridad

ver_alertas()

Muestra alertas ordenadas por fecha y estado

verificar_alertas()

Detecta stock bajo y vencimientos proximos

calcular_dias_restantes()

Estima dias de stock segun dosis y frecuencia

buscar_medicamento()

Busca un medicamento por nombre

hablar_con_ia()

Chat con asistente medico GPT-4o

main()

Menu principal con el loop del programa

Menu Principal — 10 Opciones

Estructura de navegacion del programa

Como funciona el menu

El menu se muestra en un loop (while) que se repite hasta que el usuario elige la opcion 10 (Salir). Cada opcion llama a la funcion correspondiente.

1	Agregar medicamento	Alta
2	Mostrar medicamentos	Consulta
3	Registrar toma	Accion
4	Crear alerta personalizada	Nueva
5	Ver alertas personalizadas	Consulta
6	Verificar alertas de stock y vencimiento	Alerta
7	Calcular dias restantes de stock	Calculo
8	Buscar medicamento por nombre	Consulta
9	Consultar con el asistente IA	IA
10	Salir del sistema	Salida

Condicion de salida: while opcion not in ("salir", "10") — sin uso de break, condicion directa en el while

Codigo: agregar_medicamento()

Registro de medicamentos con validaciones de fecha y stock

Validaciones implementadas

- Fecha termino no puede ser anterior a la fecha inicio (while con reingreso).
- Stock no puede ser 0 ni negativo — se repite el ingreso hasta que sea valido.
- Si el medicamento ya vencio al cargarlo: el sistema avisa pero no bloquea.
- Frecuencia se ingresa como cantidad de tomas diarias (ej: 3 si es cada 8 horas).

```
def    agregar_medicamento():
    nombre = input("Ingrese el nombre del medicamento: ")
    dosis = int(input("Ingrese la cantidad de pastillas/ml por cada toma: "))
    frecuencia = int(input("Ingrese la cantidad de tomas diarias: "))

    fecha_inicio = datetime.strptime(input("Fecha de inicio (YYYY-MM-DD): "), "%Y-%m-%d")
    fecha_termino = datetime.strptime(input("Fecha de termino (YYYY-MM-DD): "), "%Y-%m-%d")
    while    fecha_termino < fecha_inicio:
        print(    "Error: la fecha de termino no puede ser anterior al inicio.")
        fecha_termino = datetime.strptime(input("Fecha de termino: "), "%Y-%m-%d")

    stock = int(input("Stock total disponible: "))
    while    stock <= 0:
        stock = int(input("Stock no puede ser 0 o menor. Ingrese el stock: "))

    vencimiento = datetime.strptime(input("Fecha de vencimiento (YYYY-MM-DD): "), "%Y-%m-%d")
    if    vencimiento <= datetime.now():
        print(    "El medicamento ya ha vencido. No emitira alertas de vencimiento.")

    nombre_doctor = input("Nombre del doctor que receto el medicamento: ")
    medicamento = {
        "nombre": nombre, "dosis": dosis, "frecuencia": frecuencia,
        "fecha_inicio": fecha_inicio, "fecha_termino": fecha_termino,
        "stock": stock, "vencimiento": vencimiento, "nombre_doctor": nombre_doctor
    }

    return    medicamento
```

Codigo: ingresar_alerta() y ver_alertas()

Sistema de alertas personalizadas con fecha, hora y prioridad

Como funcionan las alertas personalizadas

- El usuario ingresa un mensaje descriptivo, fecha, hora opcional y prioridad (Alta/Media/Baja).
- Las alertas se guardan en la lista global `alertas_personalizadas` como diccionarios.
- `ver_alertas()` las ordena por fecha y muestra su estado: HOY, PROXIMA, PENDIENTE o VENCIDA.
- La fecha se valida con `try/except`: si el formato es incorrecto, pide de nuevo (sin break).

```
def ingresar_alerta():
    mensaje = input("Descripcion de la alerta: ").strip()
    if not mensaje:
        print("La alerta no puede estar vacia.")
        return

    # Validacion de fecha sin break
    fecha = None
    while fecha is None:
        try:
            fecha_str = input("Fecha (YYYY-MM-DD): ")
            fecha = datetime.strptime(fecha_str, "%Y-%m-%d")
        except ValueError:
            print("Formato incorrecto. Use YYYY-MM-DD")

    hora_str = input("Hora (HH:MM) o Enter: ").strip()
    hora = datetime.strptime(hora_str, "%H:%M").strftime("%H:%M")
    if hora_str else None

    prioridad = ""
    while prioridad not in ("1", "2", "3"):
        print("1.Alta 2.Media 3.Baja")
        prioridad = input("Prioridad: ").strip()

    niveles = {"1": "Alta", "2": "Media", "3": "Baja"}
    alertas_personalizadas.append({
        "mensaje": mensaje, "fecha": fecha,
        "hora": hora, "prioridad": niveles[prioridad],
        "creada": datetime.now().strftime("%Y-%m-%d %H:%M")
    })
```

```
def ver_alertas():
    if not alertas_personalizadas:
        print("No hay alertas registradas.")
        return

    ordenadas = sorted(alertas_personalizadas,
                        key=lambda a: a['fecha'])
    hoy = datetime.now().date()

    for i, alerta in enumerate(ordenadas, start=1):
        dias_faltan = (alerta['fecha'].date() - hoy).days

        if dias_faltan < 0:
            estado = "VENCIDA"
        elif dias_faltan == 0:
            estado = "HOY"
        elif dias_faltan <= 3:
            estado = "PROXIMA"
        else:
            estado = "PENDIENTE"

        print(f"{i}. {alerta['mensaje']}")
        print(f"Fecha : {alerta['fecha']}")
        print(f"Prioridad: {alerta['prioridad']}")
        print(f"Estado : [{estado}])")
```

Codigo: verificar_alertas() y calcular_dias_restantes()

Control de stock, vencimientos y estimacion de dias

Logica de verificacion

- verificar_alertas() recibe el umbral de stock y los dias de margen como parametros desde main().
- calcular_dias_restantes() multiplica dosis x frecuencia para obtener el consumo diario total.
- Ambas recorren la lista de medicamentos con un for y comparan valores numericos directamente.

```
def verificar_alertas(medicamentos,
                     umbral_stock, dias_vencimiento):
    hoy = datetime.now()
    for med in medicamentos:

        # Alerta de stock bajo
        if med['stock'] < umbral_stock:
            print( f"Alerta: El stock de "
                  f"{med['nombre']} es menor"
                  f" a {umbral_stock}."
                  f" Stock: {med['stock']}")

        # Alerta de vencimiento proximo
        dias_faltantes = (
            med['vencimiento'] - hoy).days
        if (dias_faltantes < dias_vencimiento
            and dias_faltantes >= 0):
            print( f"Alerta: {med['nombre']} "
                  f"vence en {dias_faltantes} dias.")
```

```
def calcular_dias_restantes(medicamentos):
    for med in medicamentos:
        if med['dosis'] > 0:

            # consumo total por dia
            total_dia = (
                med['dosis'] * med['frecuencia'])

            # division entera para dias completos
            dias_restantes = (
                med['stock'] // total_dia)

            print(
                f"El medicamento {med['nombre']}"
                f" tiene stock para ~{dias_restantes}"
                f" dias.")
        else:
            print(
                f"Error en la dosis de {med['nombre']}.")
```

Los parametros umbral y dias_venc se ingresan una sola vez en main() (opcion 6) y se pasan directamente a verificar_alertas().

Esto evita pedirlos dentro de la funcion y hace el codigo mas reutilizable y facil de testear.

Codigo: hablar_con_ia()

Asistente medico basado en GPT-4o con historial de conversacion

Como funciona el asistente IA

- Usa el modelo GPT-4o de OpenAI. La API key se carga desde un archivo .env (nunca en el codigo).
- El rol de sistema limita al asistente exclusivamente al Vademecum Argentino.
- El historial completo de la conversacion se envia en cada llamada para mantener contexto.
- El loop usa while mensaje != "salir" — sin break, condicion directa en el while.

```
def hablar_con_ia():
    historial_chat = [
        {
            "role": "system",
            "content": (
                "Eres un asistente medico que responde"
                " consultas sobre medicamentos usando"
                " VADEMECUM ARGENTINO. NO DEBES HABLAR"
                " DE NADA NO RELACIONADO CON MEDICAMENTOS."
            )
        }
    ]

    print( "Chat iniciado. Escribe 'salir' para volver.")

    # Loop sin break: condicion directa en el while
    mensaje = ""
    while mensaje.lower() != "salir":
        mensaje = input("Tu: ").strip()
        if mensaje.lower() != "salir":

            # Agregar mensaje del usuario al historial
            historial_chat.append({
                "role": "user",
                "content": mensaje
            })

            # Llamada a la API con todo el historial
            response = client.chat.completions.create(
                model="gpt-4o",
                messages=historial_chat
            )
            respuesta = response.choices[0].message.content
            print( f"IA: {respuesta}")

            # Guardar respuesta en el historial
            historial_chat.append({
                "role": "assistant",
                "content": respuesta
            })

    print( "Saliendo del chat...")
```


Codigo: main() y Conclusion

Loop principal del programa y cierre del proyecto

main() — Loop principal

```
def main():
    medicamentos = []
    opcion = ""

    # Sin break: el while evalua la condicion directa
    while opcion not in ("salir", "10"):
        print( "--- MENU PRINCIPAL ---")
        print( "1. Agregar medicamento")
        print( "2. Mostrar medicamentos")
        print( "3. Registrar toma")
        print( "4. Crear alerta")
        print( "5. Ver alertas")
        print( "6. Verificar alertas de stock")
        print( "7. Calcular dias restantes")
        print( "8. Buscar medicamento")
        print( "9. Consultar asistente IA")
        print( "10. Salir")
        opcion = input("Seleccione una opcion: ")

        if opcion == "1":
            medicamentos.append(agregar_medimento())
        elif opcion == "3":
            nombre = input("Nombre del medicamento: ")
            registrar_toma(medicamentos, nombre)
        elif opcion == "6":
            umbral = int(input("Umbral de stock: "))
            dias_venc = int(input("Dias de margen: "))
            verificar_alertas(medicamentos, umbral, dias_venc)
        elif opcion == "9":
            hablar_con_ia()
        elif opcion == "10":
            print( "Saliendo del sistema...")
            opcion = "salir" # termina el while

if __name__ == "__main__":
```

main()

Conclusion

Este proyecto demuestra como los conceptos basicos de programacion pueden aplicarse para resolver un problema concreto y de alto impacto en la vida cotidiana. La aplicacion de Gestion de Medicamentos no solo cumple con los requisitos tecnicos del Trabajo Practico, sino que representa una solucion genuinamente util para cualquier persona que necesite llevar un registro organizado de su medicacion.

A traves del uso de listas, funciones y estructuras de control, el programa logra gestionar informacion de manera eficiente, validar los datos ingresados y generar alertas preventivas que pueden marcar una diferencia real en la salud de los usuarios.

Integrantes del grupo

Cristian Aostri Lopez

David Espinosa

Facundo Balmacena

Patricio Zermo

UADE — Introduccion a la Algoritmia 2025